Lab Sheet 3 - Solution

SQL Functions

Objective: To practice number, aggregate, character, conversion and date functions in MySQL.

Note: Every query was executed and the output shown is the real result, on a MySQL-compatible server (MariaDB 10.11). The functions used are standard MySQL ones. Q21 and Q23 depend on the system date, so your own output will show your current date instead of the one in this PDF.

Setup (run first)

Create the database

SQL

```
DROP DATABASE IF EXISTS CollegeDB;
CREATE DATABASE CollegeDB;
```

Output

```
Query OK, 7 rows affected

Query OK, 1 row affected
```

Create tables and load sample data

SQL

```
CREATE TABLE Student (
    Student_ID INT PRIMARY KEY,
    First_Name VARCHAR(30),
    Last_Name  VARCHAR(30),
    Name       VARCHAR(60),
    Age        INT,
    Course     VARCHAR(30),
    City       VARCHAR(30)
);
CREATE TABLE Employee (
    Emp_ID    INT PRIMARY KEY,
    Emp_Name  VARCHAR(50),
    Department VARCHAR(30),
    Salary    DECIMAL(10,2),
    Join_Date DATE
);
INSERT INTO Student VALUES
    (101, 'Rahul',  'Sharma', 'Rahul Sharma', 20, 'B.Tech', 'Meerut'),
    (102, 'Priya',  'Singh',  'Priya Singh',  21, 'BCA',    'Delhi'),
    (103, 'Amit',   'Verma',  'Amit Verma',   22, 'MCA',    'Roorkee'),
    (104, 'Neha',   'Gupta',  'Neha Gupta',   19, 'B.Sc',   'Meerut'),
    (105, 'Karan',  'Mehta',  'Karan Mehta',  23, 'B.Tech', 'Dehradun'),
    (106, 'Sneha',  'Joshi',  'Sneha Joshi',  20, 'BBA',    'Haridwar');
INSERT INTO Employee VALUES
    (1, 'Arun Kapoor',  'IT',      55250.75, '2020-06-15'),
    (2, 'Meena Iyer',   'HR',      42500.40, '2019-03-01'),
    (3, 'Suresh Patel', 'Finance', 48750.50, '2021-09-20'),
    (4, 'Divya Menon',  'IT',      61200.25, '2018-12-05'),
    (5, 'Ravi Das',     'Sales',   33300.90, '2022-01-10');
SELECT * FROM Student;
SELECT * FROM Employee;
```

Output

```
Query OK, 0 rows affected

Query OK, 0 rows affected

Query OK, 6 rows affected
Records: 6  Duplicates: 0  Warnings: 0

Query OK, 5 rows affected
Records: 5  Duplicates: 0  Warnings: 0

+------------+------------+-----------+--------------+------+--------+----------+
| Student_ID | First_Name | Last_Name | Name         | Age  | Course | City     |
+------------+------------+-----------+--------------+------+--------+----------+
|        101 | Rahul      | Sharma    | Rahul Sharma |   20 | B.Tech | Meerut   |
|        102 | Priya      | Singh     | Priya Singh  |   21 | BCA    | Delhi    |
|        103 | Amit       | Verma     | Amit Verma   |   22 | MCA    | Roorkee  |
|        104 | Neha       | Gupta     | Neha Gupta   |   19 | B.Sc   | Meerut   |
|        105 | Karan      | Mehta     | Karan Mehta  |   23 | B.Tech | Dehradun |
|        106 | Sneha      | Joshi     | Sneha Joshi  |   20 | BBA    | Haridwar |
+------------+------------+-----------+--------------+------+--------+----------+
6 rows in set

+--------+--------------+------------+----------+------------+
| Emp_ID | Emp_Name     | Department | Salary   | Join_Date  |
+--------+--------------+------------+----------+------------+
|      1 | Arun Kapoor  | IT         | 55250.75 | 2020-06-15 |
|      2 | Meena Iyer   | HR         | 42500.40 | 2019-03-01 |
|      3 | Suresh Patel | Finance    | 48750.50 | 2021-09-20 |
|      4 | Divya Menon  | IT         | 61200.25 | 2018-12-05 |
|      5 | Ravi Das     | Sales      | 33300.90 | 2022-01-10 |
+--------+--------------+------------+----------+------------+
5 rows in set
```

A. Number Functions

Q1. Demonstrate ABS() with suitable examples.

SQL

```
SELECT ABS(-25) AS abs_of_negative, ABS(25) AS abs_of_positive, ABS(-7.5) AS abs_of_decimal;
```

Output

```
+-----------------+-----------------+----------------+
| abs_of_negative | abs_of_positive | abs_of_decimal |
+-----------------+-----------------+----------------+
|              25 |              25 |            7.5 |
+-----------------+-----------------+----------------+
1 row in set
```

Q2. Demonstrate ROUND() with suitable examples.

SQL

```
SELECT ROUND(123.456, 2) AS two_decimals,
       ROUND(123.456)    AS nearest_int,
       ROUND(123.456, -1) AS nearest_ten,
       ROUND(-2.6)       AS negative_value;
```

Output

```
+--------------+-------------+-------------+----------------+
| two_decimals | nearest_int | nearest_ten | negative_value |
+--------------+-------------+-------------+----------------+
|       123.46 |         123 |         120 |             -3 |
+--------------+-------------+-------------+----------------+
1 row in set
```

Q3. Demonstrate CEIL() and FLOOR().

SQL

```
SELECT CEIL(4.2) AS ceil_pos, FLOOR(4.8) AS floor_pos,
       CEIL(-4.2) AS ceil_neg, FLOOR(-4.8) AS floor_neg;
```

Output

```
+----------+-----------+----------+-----------+
| ceil_pos | floor_pos | ceil_neg | floor_neg |
+----------+-----------+----------+-----------+
|        5 |         4 |       -4 |        -5 |
+----------+-----------+----------+-----------+
1 row in set
```

Q4. Demonstrate MOD() with suitable examples.

SQL

```
SELECT MOD(17, 5) AS mod_17_5, 17 % 5 AS using_percent, MOD(10, 2) AS even_check;
-- odd/even student IDs
SELECT Student_ID, Name, MOD(Student_ID, 2) AS remainder FROM Student;
```

Output

```
+----------+---------------+------------+
| mod_17_5 | using_percent | even_check |
+----------+---------------+------------+
|        2 |             2 |          0 |
+----------+---------------+------------+
1 row in set

+------------+--------------+-----------+
| Student_ID | Name         | remainder |
+------------+--------------+-----------+
|        101 | Rahul Sharma |         1 |
|        102 | Priya Singh  |         0 |
|        103 | Amit Verma   |         1 |
|        104 | Neha Gupta   |         0 |
|        105 | Karan Mehta  |         1 |
|        106 | Sneha Joshi  |         0 |
+------------+--------------+-----------+
6 rows in set
```

Q5. Calculate the rounded salary of employees.

SQL

```
SELECT Emp_Name, Salary,
       ROUND(Salary)     AS Rounded_Salary,
       ROUND(Salary, -2) AS Rounded_to_Hundreds
FROM Employee;
```

Output

```
+--------------+----------+----------------+---------------------+
| Emp_Name     | Salary   | Rounded_Salary | Rounded_to_Hundreds |
+--------------+----------+----------------+---------------------+
| Arun Kapoor  | 55250.75 |          55251 |               55300 |
| Meena Iyer   | 42500.40 |          42500 |               42500 |
| Suresh Patel | 48750.50 |          48751 |               48800 |
| Divya Menon  | 61200.25 |          61200 |               61200 |
| Ravi Das     | 33300.90 |          33301 |               33300 |
+--------------+----------+----------------+---------------------+
5 rows in set
```

B. Aggregate Functions

Q6. Find the total number of students using COUNT().

SQL

```
SELECT COUNT(*) AS Total_Students FROM Student;
```

Output

```
+----------------+
| Total_Students |
+----------------+
|              6 |
+----------------+
1 row in set
```

Q7. Find the total salary of all employees using SUM().

SQL

```
SELECT SUM(Salary) AS Total_Salary FROM Employee;
```

Output

```
+--------------+
| Total_Salary |
+--------------+
|    241002.80 |
+--------------+
1 row in set
```

Q8. Find the average salary using AVG().

SQL

```
SELECT AVG(Salary) AS Average_Salary, ROUND(AVG(Salary), 2) AS Average_Rounded FROM Employee;
```

Output

```
+----------------+-----------------+
| Average_Salary | Average_Rounded |
+----------------+-----------------+
|   48200.560000 |        48200.56 |
+----------------+-----------------+
1 row in set
```

Q9. Find the highest salary using MAX().

SQL

```
SELECT MAX(Salary) AS Highest_Salary FROM Employee;
```

Output

```
+----------------+
| Highest_Salary |
+----------------+
|       61200.25 |
+----------------+
1 row in set
```

Q10. Find the lowest salary using MIN().

SQL

```
SELECT MIN(Salary) AS Lowest_Salary FROM Employee;
```

Output

```
+---------------+
| Lowest_Salary |
+---------------+
|      33300.90 |
+---------------+
1 row in set
```

C. Character Functions

Q11. Demonstrate UPPER() on student names.

SQL

```
SELECT Name, UPPER(Name) AS Upper_Name FROM Student;
```

Output

```
+--------------+--------------+
| Name         | Upper_Name   |
+--------------+--------------+
| Rahul Sharma | RAHUL SHARMA |
| Priya Singh  | PRIYA SINGH  |
| Amit Verma   | AMIT VERMA   |
| Neha Gupta   | NEHA GUPTA   |
| Karan Mehta  | KARAN MEHTA  |
| Sneha Joshi  | SNEHA JOSHI  |
+--------------+--------------+
6 rows in set
```

Q12. Demonstrate LOWER() on student names.

SQL

```
SELECT Name, LOWER(Name) AS Lower_Name FROM Student;
```

Output

```
+--------------+--------------+
| Name         | Lower_Name   |
+--------------+--------------+
| Rahul Sharma | rahul sharma |
| Priya Singh  | priya singh  |
| Amit Verma   | amit verma   |
| Neha Gupta   | neha gupta   |
| Karan Mehta  | karan mehta  |
| Sneha Joshi  | sneha joshi  |
+--------------+--------------+
6 rows in set
```

Q13. Demonstrate LENGTH() on student names.

SQL

```
SELECT Name, LENGTH(Name) AS Name_Length FROM Student;
```

Output

```
+--------------+-------------+
| Name         | Name_Length |
+--------------+-------------+
| Rahul Sharma |          12 |
| Priya Singh  |          11 |
| Amit Verma   |          10 |
| Neha Gupta   |          10 |
| Karan Mehta  |          11 |
| Sneha Joshi  |          11 |
+--------------+-------------+
6 rows in set
```

Q14. Demonstrate CONCAT() using first and last names.

SQL

```
SELECT First_Name, Last_Name, CONCAT(First_Name, ' ', Last_Name) AS Full_Name FROM Student;
```

Output

```
+------------+-----------+--------------+
| First_Name | Last_Name | Full_Name    |
+------------+-----------+--------------+
| Rahul      | Sharma    | Rahul Sharma |
| Priya      | Singh     | Priya Singh  |
| Amit       | Verma     | Amit Verma   |
| Neha       | Gupta     | Neha Gupta   |
| Karan      | Mehta     | Karan Mehta  |
| Sneha      | Joshi     | Sneha Joshi  |
+------------+-----------+--------------+
6 rows in set
```

Q15. Demonstrate SUBSTRING()/SUBSTR() with suitable examples.

SQL

```
SELECT Name,
       SUBSTRING(Name, 1, 4) AS first_4_chars,
       SUBSTR(Name, 3, 5)    AS from_pos3_len5,
       SUBSTRING(Name, -3)   AS last_3_chars
FROM Student;
```

Output

```
+--------------+---------------+----------------+--------------+
| Name         | first_4_chars | from_pos3_len5 | last_3_chars |
+--------------+---------------+----------------+--------------+
| Rahul Sharma | Rahu          | hul S          | rma          |
| Priya Singh  | Priy          | iya S          | ngh          |
| Amit Verma   | Amit          | it Ve          | rma          |
| Neha Gupta   | Neha          | ha Gu          | pta          |
| Karan Mehta  | Kara          | ran M          | hta          |
| Sneha Joshi  | Sneh          | eha J          | shi          |
+--------------+---------------+----------------+--------------+
6 rows in set
```

D. Conversion Functions

Q16. Convert a number into a character/string value.

SQL

```
SELECT Emp_Name, Salary,
       CAST(Salary AS CHAR)        AS Salary_As_Text,
       CONCAT('Rs. ', Salary)      AS Salary_With_Prefix
FROM Employee;
```

Output

```
+--------------+----------+----------------+--------------------+
| Emp_Name     | Salary   | Salary_As_Text | Salary_With_Prefix |
+--------------+----------+----------------+--------------------+
| Arun Kapoor  | 55250.75 | 55250.75       | Rs. 55250.75       |
| Meena Iyer   | 42500.40 | 42500.40       | Rs. 42500.40       |
| Suresh Patel | 48750.50 | 48750.50       | Rs. 48750.50       |
| Divya Menon  | 61200.25 | 61200.25       | Rs. 61200.25       |
| Ravi Das     | 33300.90 | 33300.90       | Rs. 33300.90       |
+--------------+----------+----------------+--------------------+
5 rows in set
```

Q17. Convert a string into a numeric value.

SQL

```
SELECT '123' + 7                    AS implicit_conversion,
       CAST('45.67' AS DECIMAL(10,2)) AS string_to_decimal,
       CAST('99' AS SIGNED)         AS string_to_int,
       '25' * '4'                   AS text_times_text;
```

Output

```
+--------------------+------------------+---------------+-----------------+
| implicit_conversion | string_to_decimal | string_to_int | text_times_text |
+--------------------+------------------+---------------+-----------------+
|                 130 |            45.67 |            99 |             100 |
+--------------------+------------------+---------------+-----------------+
1 row in set
```

Q18. Convert a date value into a different format.

SQL

```
SELECT Join_Date,
       DATE_FORMAT(Join_Date, '%d-%m-%Y')     AS dd_mm_yyyy,
       DATE_FORMAT(Join_Date, '%d %M %Y')     AS day_monthname_year,
       DATE_FORMAT(Join_Date, '%W, %d %b %Y') AS weekday_style
FROM Employee;
```

Output

```
+------------+------------+--------------------+------------------------+
| Join_Date  | dd_mm_yyyy | day_monthname_year | weekday_style          |
+------------+------------+--------------------+------------------------+
| 2020-06-15 | 15-06-2020 | 15 June 2020       | Monday, 15 Jun 2020    |
| 2019-03-01 | 01-03-2019 | 01 March 2019      | Friday, 01 Mar 2019    |
| 2021-09-20 | 20-09-2021 | 20 September 2021  | Monday, 20 Sep 2021    |
| 2018-12-05 | 05-12-2018 | 05 December 2018   | Wednesday, 05 Dec 2018 |
| 2022-01-10 | 10-01-2022 | 10 January 2022    | Monday, 10 Jan 2022    |
+------------+------------+--------------------+------------------------+
5 rows in set
```

Q19. Demonstrate CAST() with suitable examples.

SQL

```
SELECT CAST('2026-10-13' AS DATE)              AS to_date,
       CAST('2026-10-13 14:30:00' AS DATETIME) AS to_datetime,
       CAST(65 AS CHAR)                        AS to_char,
       CAST('250' AS UNSIGNED)                 AS to_unsigned,
       CAST(Salary AS CHAR) AS salary_text
FROM Employee LIMIT 2;
```

Output

```
+------------+---------------------+---------+-------------+-------------+
| to_date    | to_datetime         | to_char | to_unsigned | salary_text |
+------------+---------------------+---------+-------------+-------------+
| 2026-10-13 | 2026-10-13 14:30:00 | 65      |         250 | 55250.75    |
| 2026-10-13 | 2026-10-13 14:30:00 | 65      |         250 | 42500.40    |
+------------+---------------------+---------+-------------+-------------+
2 rows in set
```

Q20. Demonstrate CONVERT() where supported by the DBMS.

SQL

```
SELECT CONVERT(123, CHAR)             AS num_to_char,
       CONVERT('456', SIGNED)         AS char_to_int,
       CONVERT('2026-10-13', DATE)    AS char_to_date,
       CONVERT('Hello' USING utf8mb4) AS charset_convert;
```

Output

```
+-------------+-------------+--------------+-----------------+
| num_to_char | char_to_int | char_to_date | charset_convert |
+-------------+-------------+--------------+-----------------+
| 123         |         456 | 2026-10-13   | Hello           |
+-------------+-------------+--------------+-----------------+
1 row in set
```

E. Date Functions

Q21. Display the current date using a suitable date function.

SQL

```
SELECT CURDATE() AS today, CURRENT_DATE() AS also_today, NOW() AS date_and_time;
```

Output

```
+------------+------------+---------------------+
| today      | also_today | date_and_time       |
+------------+------------+---------------------+
| 2026-10-12 | 2026-10-12 | 2026-10-12 20:25:41 |
+------------+------------+---------------------+
1 row in set
```

Q22. Extract the year, month and day from a date.

SQL

```
SELECT Emp_Name, Join_Date,
       YEAR(Join_Date)      AS Year_Part,
       MONTH(Join_Date)     AS Month_Part,
       DAY(Join_Date)       AS Day_Part,
       MONTHNAME(Join_Date) AS Month_Name
FROM Employee;
```

Output

```
+--------------+------------+-----------+------------+----------+------------+
| Emp_Name     | Join_Date  | Year_Part | Month_Part | Day_Part | Month_Name |
+--------------+------------+-----------+------------+----------+------------+
| Arun Kapoor  | 2020-06-15 |      2020 |          6 |       15 | June       |
| Meena Iyer   | 2019-03-01 |      2019 |          3 |        1 | March      |
| Suresh Patel | 2021-09-20 |      2021 |          9 |       20 | September  |
| Divya Menon  | 2018-12-05 |      2018 |         12 |        5 | December   |
| Ravi Das     | 2022-01-10 |      2022 |          1 |       10 | January    |
+--------------+------------+-----------+------------+----------+------------+
5 rows in set
```

Q23. Find the difference between two dates.

SQL

```
SELECT DATEDIFF('2026-12-31', '2026-10-13') AS days_between;
SELECT Emp_Name, Join_Date,
       DATEDIFF(CURDATE(), Join_Date)              AS Days_In_Company,
       TIMESTAMPDIFF(YEAR, Join_Date, CURDATE())   AS Years_In_Company
FROM Employee;
```

Output

```
+--------------+
| days_between |
+--------------+
|           79 |
+--------------+
1 row in set

+--------------+------------+-----------------+------------------+
| Emp_Name     | Join_Date  | Days_In_Company | Years_In_Company |
+--------------+------------+-----------------+------------------+
| Arun Kapoor  | 2020-06-15 |            2310 |                6 |
| Meena Iyer   | 2019-03-01 |            2782 |                7 |
| Suresh Patel | 2021-09-20 |            1848 |                5 |
| Divya Menon  | 2018-12-05 |            2868 |                7 |
| Ravi Das     | 2022-01-10 |            1736 |                4 |
+--------------+------------+-----------------+------------------+
5 rows in set
```

Q24. Add a specified number of days to a date.

SQL

```
SELECT Emp_Name, Join_Date,
       DATE_ADD(Join_Date, INTERVAL 30 DAY) AS Plus_30_Days,
       Join_Date + INTERVAL 90 DAY          AS Probation_End,
       DATE_SUB(Join_Date, INTERVAL 7 DAY)  AS Minus_7_Days
FROM Employee;
```

Output

```
+--------------+------------+--------------+---------------+--------------+
| Emp_Name     | Join_Date  | Plus_30_Days | Probation_End | Minus_7_Days |
+--------------+------------+--------------+---------------+--------------+
| Arun Kapoor  | 2020-06-15 | 2020-07-15   | 2020-09-13    | 2020-06-08   |
| Meena Iyer   | 2019-03-01 | 2019-03-31   | 2019-05-30    | 2019-02-22   |
| Suresh Patel | 2021-09-20 | 2021-10-20   | 2021-12-19    | 2021-09-13   |
| Divya Menon  | 2018-12-05 | 2019-01-04   | 2019-03-05    | 2018-11-28   |
| Ravi Das     | 2022-01-10 | 2022-02-09   | 2022-04-10    | 2022-01-03   |
+--------------+------------+--------------+---------------+--------------+
5 rows in set
```

Q25. Display employees' joining dates in a required date format.

SQL

```
SELECT Emp_Name,
       DATE_FORMAT(Join_Date, '%d/%m/%Y')   AS Joining_Date,
       DATE_FORMAT(Join_Date, '%d %b, %Y')  AS Joining_Date_Short
FROM Employee
ORDER BY Join_Date;
```

Output

```
+--------------+--------------+--------------------+
| Emp_Name     | Joining_Date | Joining_Date_Short |
+--------------+--------------+--------------------+
| Divya Menon  | 05/12/2018   | 05 Dec, 2018       |
| Meena Iyer   | 01/03/2019   | 01 Mar, 2019       |
| Arun Kapoor  | 15/06/2020   | 15 Jun, 2020       |
| Suresh Patel | 20/09/2021   | 20 Sep, 2021       |
| Ravi Das     | 10/01/2022   | 10 Jan, 2022       |
+--------------+--------------+--------------------+
5 rows in set
```